

Experiment 3: Selection Sort – Library Book Reordering

Aim

To implement the **Selection Sort** algorithm in Python to rearrange library books by their **ID numbers** and count the number of **physical moves (swaps)** required.

Problem Statement

A librarian wants to rearrange a shelf of books according to their **ID numbers**. Moving a book (swap) is time-consuming, while comparing book IDs is fast. Use **Selection Sort** to sort the books and report the number of physical moves (swaps).

Algorithm

Step 1: Start.

Step 2: Read the number of book IDs.

Step 3: Read the book IDs into an array.

Step 4: Initialize moves = 0.

Step 5: For each position in the array:

- Find the smallest book ID in the remaining unsorted part.
- If it is not already in the correct position, swap it.
- Increment the move counter.

Step 6: Display the sorted book IDs.

Step 7: Display the total number of physical moves.

Step 8: Stop.

Source Code (Python)

```
main.py  
1  # Library Book Reordering using Selection Sort
2
3  n = int(input("Enter number of books: "))
4
5  books = []
6
7  print("Enter Book IDs:")
8  for i in range(n):
9      books.append(int(input()))
10
11  moves = 0
12
13  for i in range(n - 1):
14      min_index = i
15
16      for j in range(i + 1, n):
17          if books[j] < books[min_index]:
18              min_index = j
19
20      if min_index != i:
21          books[i], books[min_index] = books[min_index], books[i]
22          moves += 1
23
24  print("Sorted Book IDs:", books)
25  print("Physical Moves:", moves)
```

Output

```
Output
Enter number of books: 7
Enter Book IDs:
305
102
250
118
199
400
101
Sorted Book IDs: [101, 102, 118, 199, 250, 305, 400]
Physical Moves: 4

=== Code Execution Successful ===
```

Complexity Analysis

Aspect	Complexity
Time Complexity	$O(n^2)$
Space Complexity	$O(1)$

Maximum Physical Moves $\leq n - 1$

Result

The Selection Sort algorithm was successfully implemented to reorder library books by their ID numbers. The program correctly sorted the book IDs and displayed the number of physical moves (swaps) required, demonstrating that Selection Sort minimizes the number of swaps.